

Terrain Manipulation System — Implementation Deep-Dive

Status: Partially implemented. Data definitions, CharacterController state tracking, AI activation trigger, and combat log output are all in place. The actual grid/movement integration (marking cells as difficult terrain, visual highlights, duration ticking, damage-over-time) is **stub/placeholder** — the system logs its intent but does not yet affect gameplay.

1. TerrainManipulationDefinition — Class Structure

File: Assets/Scripts/Character/Creatures/NPCDatabase.cs (line 409)

```
public class TerrainManipulationDefinition
{
    public string Name = "Ground Manipulation";
    public int RadiusFeet = 10;           // Affected area radius in feet
    public TerrainEffectType EffectType = TerrainEffectType.DifficultTerrain;
    public int DamagePerRound =
0;           // 0 = no damage, X = X damage each round standing in it
    public DamageType DamageType = DamageType.Piercing;
    public int SetupRounds = 0;           // 0 = instant, X = X rounds to manifest
    public int DurationRounds = 10;       // How long effect persists
    public bool FollowsCaster = true;      // true = centered on caster, moves with
them                                           // false = fixed at cast location

    public TerrainManipulationDefinition Clone()
    {
        return (TerrainManipulationDefinition)MemberwiseClone();
    }
}
```

Companion Enum

```
public enum TerrainEffectType
{
    DifficultTerrain, // Doubles movement cost per square
    Dangerous,        // Deals damage each round (uses DamagePerRound)
    Slippery           // Balance checks or fall prone (not yet implemented)
}
```

Property Summary

Property	Type	Purpose	Default
Name	string	Display name for combat log	"Ground Manipulation"
RadiusFeet	int	Radius of affected area in feet (÷5 for squares)	10
EffectType	TerrainEffectType	What kind of terrain hazard	DifficultTerrain
DamagePerRound	int	Flat damage dealt each round to creatures standing in the area	0
DamageType	DamageType	Type of damage (Piercing, Fire, Acid, etc.)	Piercing
SetupRounds	int	Rounds before the effect activates (0 = instant)	0
DurationRounds	int	How many rounds the effect persists	10
FollowsCaster	bool	Whether the area moves with the caster or stays fixed	true

2. Gibbering Moulder's Configuration

File: Assets/Scripts/Character/Creatures/NPCDatabase_G.cs (line 738)

```

TerrainManipulation = new TerrainManipulationDefinition
{
    Name = "Ground Manipulation",
    RadiusFeet = 10, // 2-square radius around the moulder
    EffectType = TerrainEffectType.DifficultTerrain,
    DamagePerRound = 0, // No damage, just slows movement
    SetupRounds = 0, // Instant activation
    DurationRounds = 10, // Lasts 10 rounds
    FollowsCaster = true // Moves with the moulder
},

```

D&D 3.5e Rules Reference (MM p.126)

Ground Manipulation (Su): At will, as a standard action, a gibbering moulder can cause the ground within 10 feet to become soft and spongy. All creatures in the area must make a Reflex save (DC 13) or sink partway into the ground, becoming entangled. The moulder can move normally through the affected area.

Current configuration notes:

- The definition captures the radius (10 ft) and terrain type (difficult terrain) correctly.
- **Missing from definition:** Reflex save DC, entangled condition on failure, caster immunity.
- `FollowsCaster = true` reflects the moulder's continuous ground-softening effect.
- `DamagePerRound = 0` is correct — the ability doesn't deal direct damage.

3. CharacterController — State Tracking

File: Assets/Scripts/Character/Controller/CharacterController.cs

Fields (line 531-534)

```
[SerializeField] private TerrainManipulationDefinition _terrainManipulation;
[SerializeField] private int _terrainManipulationDurationRemaining;
public bool HasTerrainManipulation => _terrainManipulation != null;
public TerrainManipulationDefinition GetTerrainManipulationDefinition() => _terrainMa-
nipulation;
```

Configuration (line 3795)

Called from `GameManager.NPCSetup.cs` during spawn:

```
public void ConfigureTerrainManipulation(TerrainManipulationDefinition terrainManip)
{
    _terrainManipulation = terrainManip?.Clone();
    _terrainManipulationDurationRemaining = terrainManip?.DurationRounds ?? 0;
}
```

Duration Tracking (line 3816)

```
public void TickTerrainManipulationDuration()
{
    if (_terrainManipulation != null && _terrainManipulationDurationRemaining > 0)
        _terrainManipulationDurationRemaining--;
}

public bool IsTerrainManipulationActive()
{
    return _terrainManipulation != null
        && (_terrainManipulation.FollowsCaster || _terrainManipulationDurationRemain-
ing > 0);
}
```

Spawn Pipeline

In `GameManager.NPCSetup.cs` (line 619):

```
if (def.TerrainManipulation != null)
    npc.ConfigureTerrainManipulation(def.TerrainManipulation);
```

What's Tracked vs What's Missing

Aspect	Status	Details
Definition stored on controller	✓ Implemented	Cloned and stored as <code>_terrainManipulation</code>
Duration remaining counter	✓ Implemented	<code>_terrainManipulationDurationRemaining</code> field exists
HasTerrainManipulation check	✓ Implemented	Boolean property for AI queries
IsTerrainManipulationActive()	✓ Implemented	Checks duration or follows-caster flag
TickTerrainManipulationDuration()	⚠ Defined but never called	The method exists but no turn-processing code invokes it
Affected cells tracking	✗ Not implemented	No <code>HashSet<Vector2Int></code> of affected cells
Caster immunity flag	✗ Not implemented	No way to exempt the caster from their own terrain

4. AI Activation — How Combat Triggers It

File: `Assets/Scripts/Services/AIService.cs`

When It Triggers

In `ExecuteAggressiveMeleeTurn` (line 524), terrain manipulation is activated **after the NPC moves** during its aggressive melee turn:

```
// After moving toward target...
npc.Actions.UseMoveAction();

// Activate terrain manipulation after moving closer
if (npc.HasTerrainManipulation)
    ActivateTerrainManipulation(npc);
```

This means the mouther:

1. Selects a target
2. Checks for pre-movement Spittle opportunity
3. **Moves** toward the target

4. **Activates terrain manipulation** at its new position

5. Continues to attempt melee/engulf attacks

The ActivateTerrainManipulation Method (line 3089)

```
public void ActivateTerrainManipulation(CharacterController npc)
{
    if (npc == null || npc.Stats == null || !npc.HasTerrainManipulation)
        return;

    TerrainManipulationDefinition terrain = npc.GetTerrainManipulationDefinition();
    if (terrain == null)
        return;

    _gameManager.CombatUI?.ShowCombatLog(
        CombatLogHelper.Info("", $"{npc.Stats.CharacterName} activates {terrain.Name}
" +
                                $"in a {terrain.RadiusFeet}-ft radius!"));

    // Mark terrain as difficult in the area around the NPC
    // This would typically interact with the GridSystem to mark affected cells
    // For now, we just log it and track the state
    Debug.Log($"[AI] {npc.Stats.CharacterName} activates {terrain.Name} " +
        $"around {npc.GridPosition} (radius {terrain.RadiusFeet} ft)");
}
```

5. What Actually Happens Right Now

When a gibbering moulder activates Ground Manipulation during combat:

What Players See

1. **Combat log message:** "☐ Gibbering Moulder activates Ground Manipulation in a 10-ft radius!"
2. **That's it.** No visual change on the grid, no movement penalty, no save prompt.

What the Console Logs

```
[AI] Gibbering Moulder activates Ground Manipulation around (12, 8) (radius 10 ft)
```

What Does NOT Happen

- **✗** No cells are marked as difficult terrain in `MovementService._difficultTerrainSquares`
- **✗** No visual highlight/overlay appears on affected grid cells
- **✗** No movement cost increase for characters traversing the area
- **✗** No five-foot-step blocking in the affected zone
- **✗** No Reflex save prompt for characters already standing in the area
- **✗** No entangled condition application
- **✗** No duration countdown (`TickTerrainManipulationDuration` is never called)
- **✗** No area update when the moulder moves (despite `FollowsCaster = true`)
- **✗** No damage-over-time processing (for future `Dangerous` type effects)

6. Implementation Status — What Exists vs What's Placeholder

✓ Fully Implemented

Component	Location	Details
Data definition class	<code>NPCDatabase.cs:409</code>	All properties defined with sensible defaults
Effect type enum	<code>NPCDatabase.cs:427</code>	Three effect types: Difficult-Terrain, Dangerous, Slippery
Creature configuration	<code>NPCDatabase_G.cs:738</code>	Gibbering moulder's specific values
Spawn-time setup	<code>GameManager.NPCSetup.cs:619</code>	Definition cloned onto CharacterController
Controller state storage	<code>CharacterController.cs:531</code>	Private field + public accessors
Duration tracking methods	<code>CharacterController.cs:3816</code>	Tick and active-check methods exist
AI trigger point	<code>AIService.cs:524</code>	Activation after movement in aggressive melee turn
Combat log output	<code>AIService.cs:3098</code>	Player-visible message about activation
Debug logging	<code>AIService.cs:3104</code>	Console logging for development

⚠ Defined But Not Connected

Component	Issue
<code>TickTerrainManipulationDuration()</code>	Method exists but is never called from any turn-processing code
<code>IsTerrainManipulationActive()</code>	Method exists but is never queried by any system

✗ Not Implemented (Placeholder/Missing)

Component	What's Needed
Grid cell marking	Call <code>GameManager.SetAreaDifficultTerrain()</code> with affected cells
Cell calculation	Compute which grid squares fall within <code>RadiusFeet</code> of caster position
Movement cost integration	Already exists in <code>MovementService.GetMovementCost()</code> — just needs cells to be marked
Five-foot-step blocking	Already exists in <code>MovementService.CanTake5FootStep()</code> — blocked by difficult terrain automatically
Visual highlights	Apply cell highlights (like Entangle/Grease do via <code>PersistentAreaEffect</code>)
FollowsCaster update	When mouther moves, clear old cells + mark new cells around new position
Duration countdown	Call <code>TickTerrainManipulationDuration()</code> from <code>BeginNPCTurnForAI()</code> or <code>StartNewTurn()</code>
Expiration cleanup	When duration runs out, clear difficult terrain cells and visual highlights
Reflex save on activation	Per RAW, creatures in area must save or become entangled
Caster immunity	Mouther should be able to move freely through its own terrain
Damage-over-time	For <code>TerrainEffectType.Dangerous</code> , apply <code>DamagePerRound</code> each round
Slippery terrain	For <code>TerrainEffectType.Slippery</code> , require Balance checks

7. What Would Need to Be Added for Full Functionality

Phase 1: Core Difficult Terrain (Minimum Viable)

These changes would make terrain manipulation actually affect movement:

7a. Cell Calculation Helper

```
// In AIService.cs or a utility class
private List<Vector2Int> CalculateTerrainAffectedCells(Vector2Int center, int radiusFeet)
{
    int radiusSquares = radiusFeet / 5;
    var cells = new List<Vector2Int>();
    for (int dx = -radiusSquares; dx <= radiusSquares; dx++)
    {
        for (int dy = -radiusSquares; dy <= radiusSquares; dy++)
        {
            Vector2Int cell = new Vector2Int(center.x + dx, center.y + dy);
            if (SquareGridUtils.GetDistance(center, cell) <= radiusSquares)
                cells.Add(cell);
        }
    }
    return cells;
}
```

7b. Mark Cells When Activating

Replace the placeholder in `ActivateTerrainManipulation` :

```
// Calculate affected cells
List<Vector2Int> affectedCells = CalculateTerrainAffectedCells(
    npc.GridPosition, terrain.RadiusFeet);

// Mark as difficult terrain in the movement system
_gameManager.SetAreaDifficultTerrain(affectedCells, true);
```

Immediate benefit: `MovementService.GetMovementCost()` already adds +1 cost per difficult terrain square. `CanTake5FootStep()` already blocks 5-foot steps into difficult terrain. These systems are fully implemented and just need the cells to be populated.

7c. Tick Duration Each Turn

In `GameManager.BeginNPCTurnForAI()` , add:

```
npc.TickTerrainManipulationDuration();
```

7d. FollowsCaster — Update on Movement

When the mouther moves, clear old cells and mark new ones:


```
// Before movement: clear old terrain cells
if (npc.HasTerrainManipulation && npc.IsTerrainManipulationActive())
{
    var oldCells = CalculateTerrainAffectedCells(npc.GridPosition, terrain.RadiusFeet);
    _gameManager.SetAreaDifficultTerrain(oldCells, false);
}

// After movement: mark new terrain cells
if (npc.HasTerrainManipulation && npc.IsTerrainManipulationActive())
{
    var newCells = CalculateTerrainAffectedCells(npc.GridPosition, terrain.RadiusFeet);
    _gameManager.SetAreaDifficultTerrain(newCells, true);
}
```

7e. Cleanup on Death/Expiration

When the mouther dies or terrain expires, clear marked cells:

```
_gameManager.SetAreaDifficultTerrain(affectedCells, false);
```

Phase 2: Visual Feedback

7f. Cell Highlights

Use the existing `SquareCell.SetCustomHighlight()` system to show affected area.

Reference implementation: `EntangleAreaEffect` and `GreaseAreaEffect` both do this.

```
// Brown/muddy color for ground manipulation
Color terrainColor = new Color(0.45f, 0.3f, 0.15f, 0.35f);
foreach (Vector2Int cellPos in affectedCells)
{
    SquareCell cell = grid.GetCell(cellPos);
    cell?.SetCustomHighlight(terrainColor);
}
```

Phase 3: Full D&D 3.5e Compliance

7g. Reflex Save + Entangled Condition

Per RAW, creatures in the area must make Reflex saves:

```
foreach (CharacterController target in creaturesInArea)
{
    if (target == npc) continue; // Caster immune

    SaveResult save = SpellSaveResolver.RollSave(target, SaveType.Reflex, saveDC);
    if (!save.Saved)
    {
        target.ApplyCondition(CombatConditionType.Entangled, 1, npc.Stats.CharacterName);
        // Log entangled
    }
}
```

7h. Dangerous Terrain (DamagePerRound > 0)

For future creatures with damaging terrain effects:

```
if (terrain.EffectType == TerrainEffectType.Dangerous && terrain.DamagePerRound > 0)
{
    foreach (CharacterController victim in creaturesStandingInArea)
    {
        victim.Stats.TakeDamage(terrain.DamagePerRound);
        // Log damage
    }
}
```

7i. Slippery Terrain (Balance Checks)

For future `TerrainEffectType.Slippery` :

```
// On entering or standing in slippery terrain
// Roll Balance check (DC varies) or fall prone
```

8. Existing Systems That Would “Just Work”

The project already has mature difficult terrain integration. Once cells are marked, these systems activate automatically with **zero additional code**:

System	File	What It Does
Movement cost doubling	<code>MovementService.cs:158</code>	Adds +1 cost per difficult terrain square entered
Five-foot step blocking	<code>MovementService.cs:356</code>	Blocks 5-foot steps into difficult terrain squares
Pathfinding cost	<code>SupportActions.cs:1284</code>	Combat maneuver pathfinding accounts for difficult terrain
Grid pathfinding	<code>SquareGrid.cs</code>	A* pathfinding includes terrain cost in calculations

These are the same systems used by `Entangle` , `Grease` , and `Black Tentacles` spell area effects.

9. Architecture Diagram



10. Only Creature Currently Using This System

Creature	CR	Terrain Effect	Radius	Duration	Follows	Damage
Gibbering Mouter	5	DifficultTerrain	10 ft	10 rounds	Yes	None

The system is designed generically — any future creature can define a `TerrainManipulation` block in its `NPCDefinition` and it will flow through the same pipeline.

Summary

The terrain manipulation system has a **complete data pipeline** from definition → spawn → controller storage → AI trigger → combat log output. What's missing is the **grid integration bridge** — the single step of calling `GameManager.SetAreaDifficultTerrain()` with calculated cells. The difficult terrain movement system itself is fully mature (used by Entangle, Grease, etc.) and would activate immediately once cells are populated. The estimated effort to reach minimum viable functionality (difficult terrain affecting movement) is small — primarily implementing cell calculation and wiring the activation to `SetAreaDifficultTerrain()`.